

Trellis

Your corner of the web, tile by tile.

A lightweight, self-hosted link-in-bio homepage with a drag-and-drop editor. This book collects every guide: using the editor, designing and customising pages, deploying to your server and changing the code.

Documentation for version 1.0.0 · 13 October 2026

Contents

USING TRELLIS

Introduction 3

Getting started 4

Editor guide 6

Cards 10

Design 14

Customising 18

Site name, favicon and sharing 22

Visitors, forms and statistics 23

Search engines and AI 26

RUNNING TRELLIS

Deployment 28

Configuration 32

Data and backups 35

Multiple sites 37

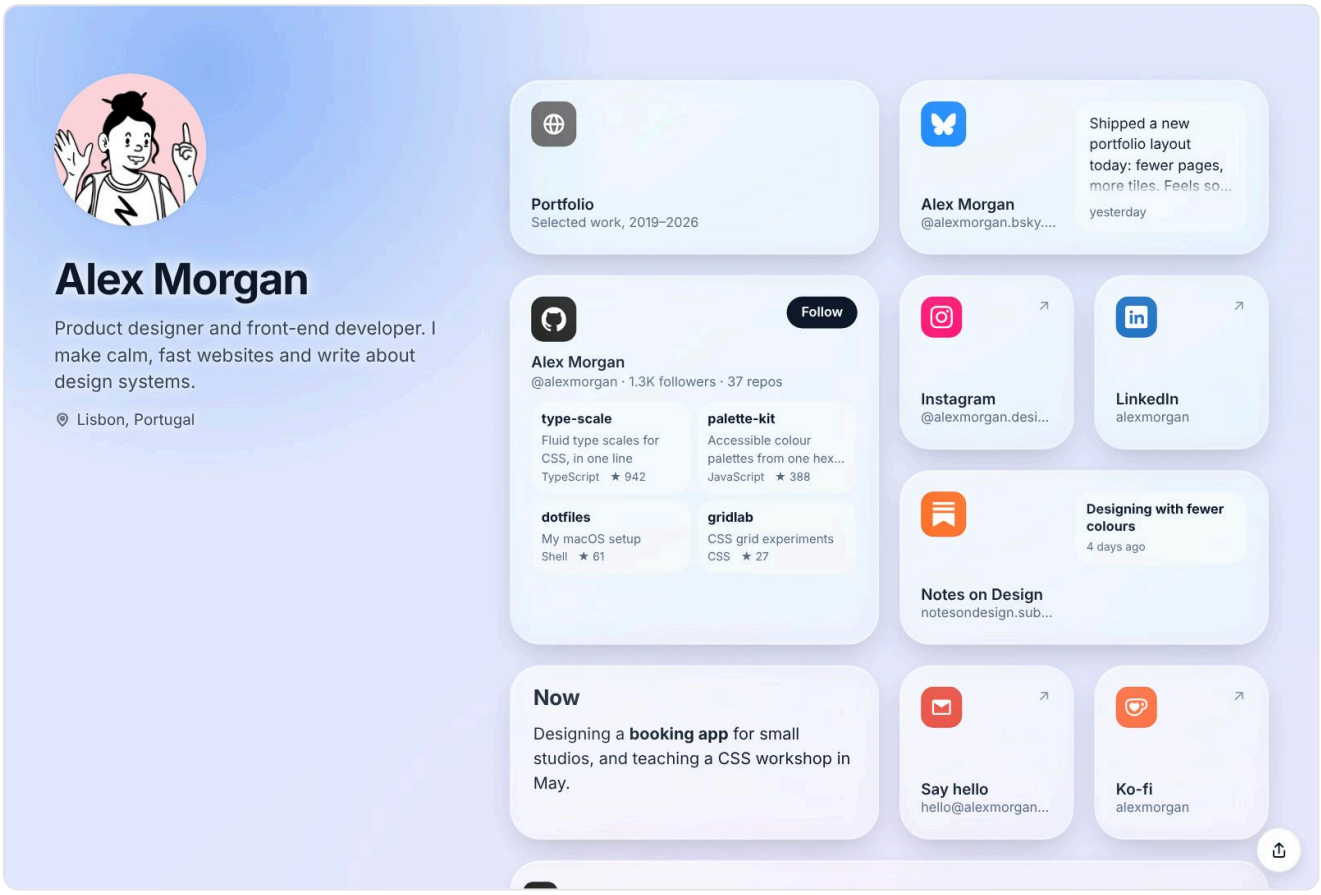
Security 40

Troubleshooting 42

CHANGING TRELLIS

Development 44

Introduction



Getting started

You need Node.js 22.9 or newer (`node -v` to check). Older versions don't have the built-in `.env` support the scripts rely on.

Install

```
git clone https://github.com/londonwebnerd/Trellis.git trellis
cd trellis
npm install
npm run set-password    # at least 8 characters
```

The editor address

There's no `/admin`. The editor lives at a random address that's created the first time the server starts:

```
npm run admin-url
#   https://<your-domain>/_3AwjOD23BIY9hTAculICL-79
```

Bookmark it and keep it to yourself. You still need the password to sign in, but the server only accepts sign-ins from that page, so bots can't even start guessing passwords. If it ever leaks, `npm run admin-url -- --rotate` makes a new one.

Running it locally

```
npm run dev
```

The page is at http://localhost:5173 and the editor is the same address with your secret path on the end. `npm run dev` runs the API on port 3001 and Vite on 5173 with hot reload.

To try the production build instead:

```
npm run build    # also makes release/trellis-<version>.zip
npm start        # http://localhost:3001
```

A first edit

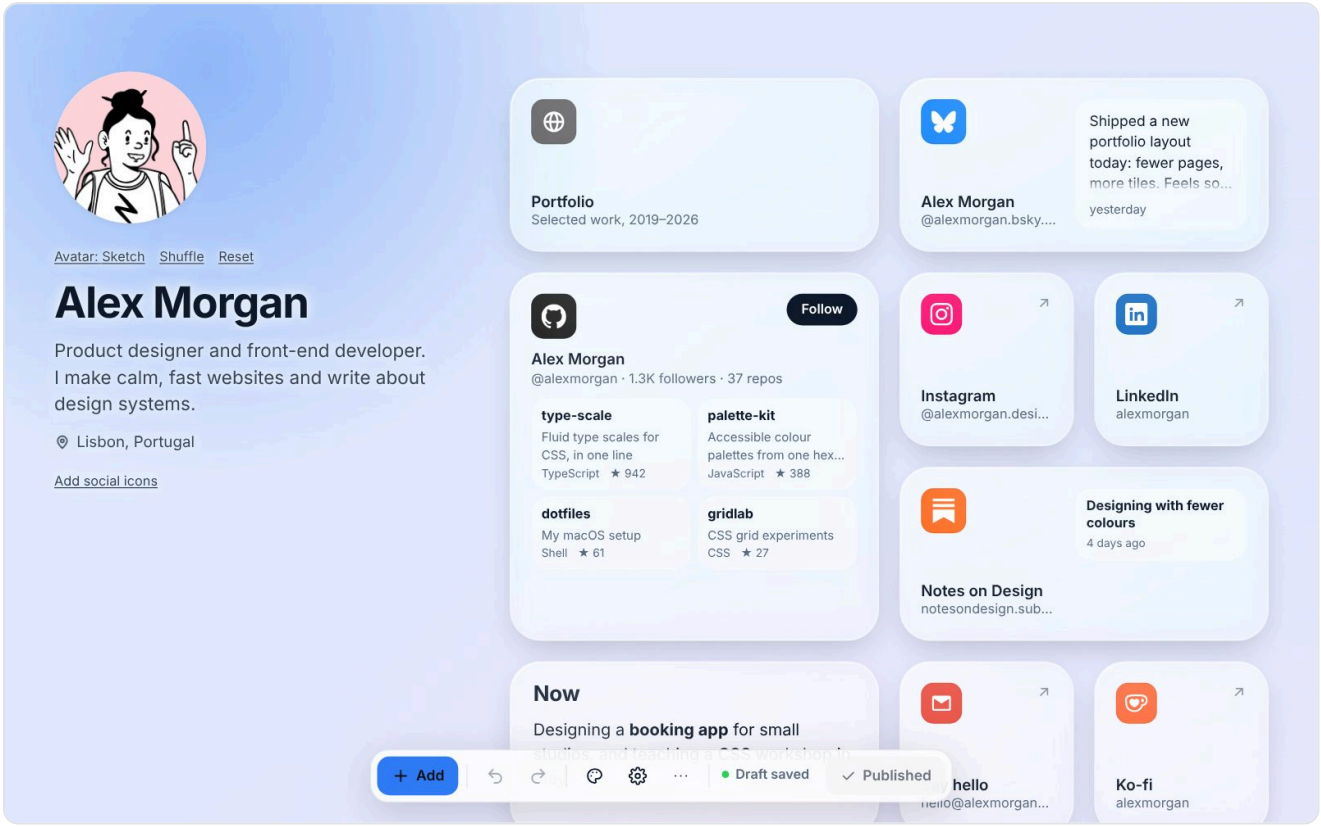
1. Open the editor and sign in.
2. Click your name, bio and location to change them, and the round picture to upload a photo.
3. Click **Add**, paste a link (your Instagram or GitHub, say) and press **Add**. Trellis picks the right card.
4. Open the palette button and try a few presets.
5. Set the site name and favicon under the gear button.
6. Press **Publish**. Until you do, visitors still see the old version.

Changes save as a draft while you work, so you can close the tab and come back later.

From here, the editor guide covers the rest of the editor, and deployment explains how to put it online.

Editor guide

Open your editor address (`npm run admin-url` if you've lost it) and sign in. You'll see the page as visitors see it, with a toolbar along the bottom.



From left to right, the toolbar has **Add**, undo and redo (⌘Z / ⇧⌘Z, or Ctrl+Z / Ctrl+Y), the Design panel (palette icon), Settings (gear), a **More** menu, the save status and **Publish**. More holds the phone preview, Statistics, Inbox, Short links, Version history, Sign-in & security, a link to the live page, Discard unpublished changes and Log out.

Drafts and publishing

Everything you change is saved as a draft straight away. Visitors keep seeing the published page until you press **Publish**, and the button says "Published" once they see exactly what you see. If you close the browser, the draft is still there next time. **More > Discard unpublished changes** throws the draft away, and you can undo that too.

If a save fails (you lost your connection, say), the status turns into "Save failed · Retry".

Settings like "Hide from search engines" also wait for Publish. Scheduled cards appear and disappear on time without you publishing again.

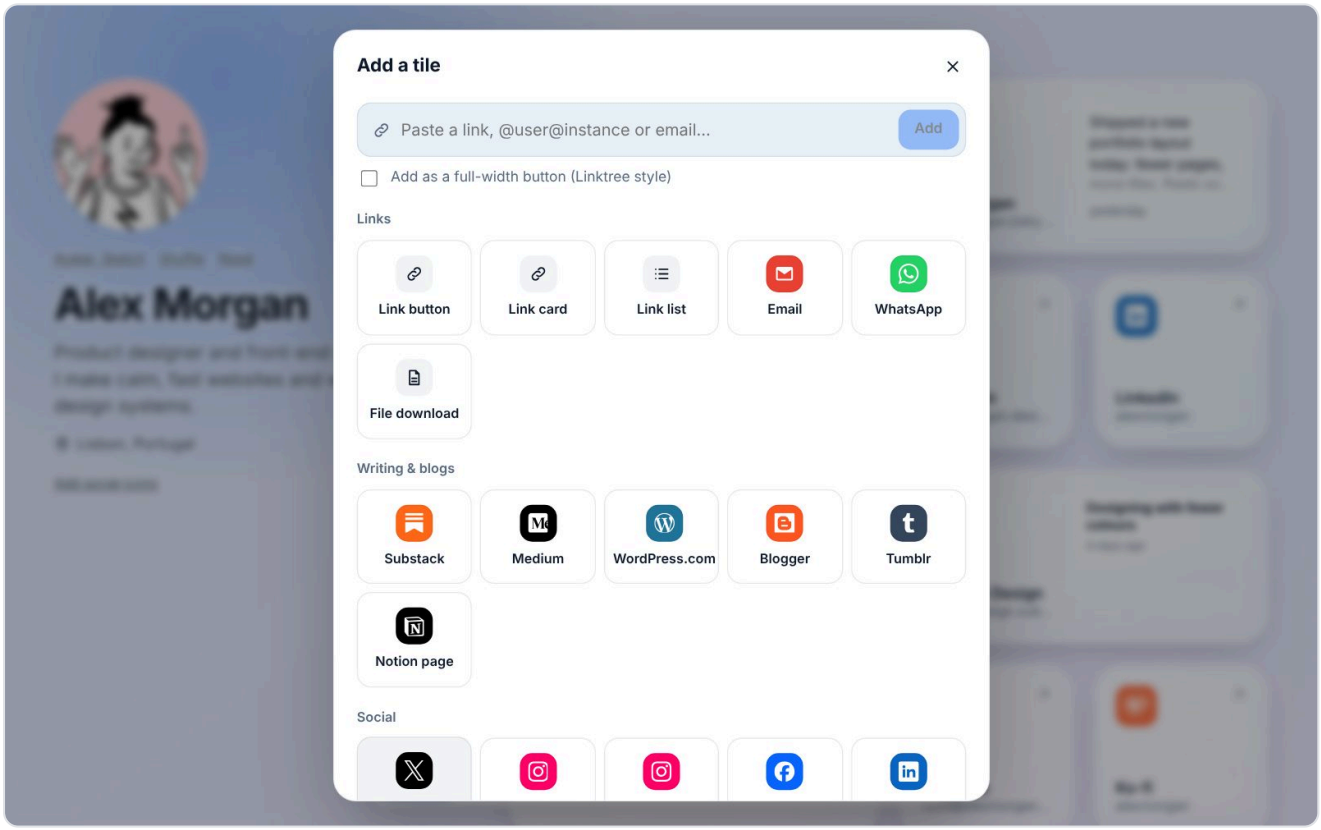
Version history

More > Version history keeps the last 50 publishes, plus an autosave roughly every ten minutes while you edit. **Restore** loads an old version into the editor. Your current version stays in the history, and nothing goes live until you publish.

Adding cards

Click **Add** and paste almost anything: a link, a Mastodon handle like `@you@mastodon.social`, or an email address. Trellis works out what it is, so a Spotify link becomes a player and a GitHub profile becomes a GitHub card. Tick **Add as a full-width button** if you want a slim Linktree-style row.

You can also pick a card from the menu. Some of them (Ko-fi, Patreon and others) start with the address filled in, so you only add your username. Cards lists them all.



Moving and resizing

Drag a card to move it, and the others make room. On a touch screen, long-press first. With the keyboard, tab to a card, press Space, use the arrow keys and press Space again to drop it.

Cards are 1 to 4 columns wide and 1 to 8 rows tall. Two rows make a square and one row is a slim button. Hover a card and pick a size at the bottom (Button, Small, Wide, Tall, Large), drag the handle in its bottom-right corner, or use the Width and Height boxes in the card's settings. Some cards have a minimum size: videos are at least 2 × 2, for example.

Bigger cards show more. A wide Bluesky card includes your latest post, and a GitHub card lists more repositories the more room it has. Phones get two columns, so anything 3 or 4 wide becomes full width.

Phones and tablets

There's no hover on a touch screen, so you tap a card to select it, which shows its buttons (duplicate, edit, delete, sizes) and the resize handle. Tap it again to open its settings, or tap empty space to deselect. On smaller screens the Design panel opens as a sheet from the bottom, and the save status shrinks to a coloured dot (tap it when it's red to retry).

Editing a card

Click a card (or the pencil on it) to open its settings. Changes show on the page behind the dialog as you make them. Deleting a card shows an Undo button for a few seconds, and everything else can be undone with ⌘Z / Ctrl+Z too.

Your profile

Click your name, bio or location to edit them, and click the picture to upload a photo (JPEG, PNG, WebP, GIF or AVIF). Photos open in a crop window first. Trellis then resizes them and saves them as WebP in the browser, which also strips the location and camera details. Big phone photos are fine; the upload is usually a few hundred KB.

Add social icons under the bio adds a row of small icons. Paste your profile links and the right icons are picked. The row can sit under the bio or at the bottom of the page.

Text and link lists

The Rich text card has a small toolbar: bold, italic, underline, strikethrough, heading, lists, quote and links (⌘K / Ctrl+K). You can paste from Word or a web page; the formatting gets cleaned up.

A Link list holds as many links as you like. Drag the handle to reorder them. Email addresses and `tel:` numbers work as well as web links.

Two tabs or two devices

If you edit in two places at once, nothing gets overwritten. When the page has been saved somewhere else since you loaded it, the editor stops and asks. **Load the other version** shows what was saved elsewhere (and Undo brings yours back), and **Keep this tab's version** saves yours over it. A tab you've left alone picks up changes from elsewhere on its own when you come back to it.

Short links

More > Short links gives you addresses like `yoursite.com/cv` that forward to anywhere you like: a CV, a booking page, a video, or a `mailto:` address. They're handy on business cards and slides.

They work as soon as you save them, without publishing, and you can change where one points whenever you like. Each one counts its visits (not yours or bots'), and they also show up in Statistics. Names the site already uses, such as `api`, `uploads` or your editor path, aren't allowed.

Signing in and security

More > Sign-in & security is where you turn on two-step sign-in. You scan a QR code with an authenticator app (1Password, Google Authenticator, Authy, Bitwarden and so on) and from then on sign in with your password plus a six-digit code. You also get ten recovery codes, so save them somewhere safe. Each one works once if you lose your phone. If you lose both, `npm run reset-2fa` on the server turns two-step sign-in off again.

The same screen lists the devices that are signed in, so you can sign out one or all of the others, and shows recent sign-in attempts, failed ones included.

Backups and export

Settings > Backup shows when the server last backed itself up and has a **Back up now** button. Restoring happens on the server; see Data and backups.

Export JSON downloads the page and design without the photos, which is handy before a big change. **Import JSON** loads one back, and you can undo that as well.

Cards

Cards (called tiles in the code) are what a page is made of. Any card can be moved, resized, duplicated or deleted; the editor guide explains how.

A card that isn't finished yet, like a file card without a file or a gallery without photos, shows a hint in the editor but stays off the live page until you fill it in.

Links

Card	What it shows	Notes
Link button	A slim, full-width row with an icon and a title	Really just a link card that's one row tall. Any card can be one row tall
Link card	Icon, title, subtitle and a button such as "Join" or "Support"	The icon, colour and button text come from the address
Link list	Several links stacked in one card	Web links, email addresses and `tel:` numbers. Two columns when it's 3 or 4 wide
Email	A `mailto:` link and a "Copy address" button	You can add a subject line that's filled in when the visitor's mail app opens
WhatsApp	A "Message me" card that opens a chat	Use the full international number, e.g. `+44 7700 900123`. It's visible on the page. You can add a starting message

Writing

Card	What it shows	Address to use
Substack	Your latest posts and "Subscribe"	`https://<you>.substack.com`
Medium	Latest posts and "Follow"	`https://medium.com/@<you>`
WordPress.com	Latest posts and "Read"	`https://<site>.wordpress.com`
Blogger	Latest posts and "Read"	`https://<blog>.blogspot.com`
Tumblr	Latest posts and "Follow"	`https://www.tumblr.com/<you>`
Notion	A link to the page, or the page itself in a tall card	To show the page inside the card, publish it to the web in Notion and use the `notion.site` link. Private pages can only be linked

The five blog services all use the Blog card. Posts come from the blog's public feed, which your server fetches.

Social networks

Card	What it shows	Notes

Twitter	Profile card; a tall card can embed one post	Listed as "X / Twitter" in the editor. Twitter doesn't allow embedding a whole timeline any more
Instagram	Profile card	
Instagram feed	Your latest posts in a grid	Needs an Instagram token, see below
Facebook	Profile or Page card; tall cards can show a Page's timeline	Timelines only work for public Pages
LinkedIn	Profile card with your headline; tall cards can embed a post	LinkedIn has no profile embed
Bluesky	Profile, followers and your latest post	
Mastodon	The same, on any server (`you@instance.social`)	

Creators

Card	Shows	Notes
GitHub	Followers, repo count and your top repositories	Paste `github.com/<you>` . Fetched from GitHub's public API
Buy Me a Coffee, Ko-fi, Patreon	Link card with a support button	
Discord	Link card with "Join"	Use an invite link that never expires
DeviantArt, Flickr	Link card	

Music and video

Card	Notes
Spotify	Playlists, albums, tracks, artists and podcasts. One row tall gives the compact player
Apple Music	In Apple Music use Share > Copy Link. One row tall gives the compact player
SoundCloud	Tracks, playlists or a profile
Bandcamp	Bandcamp needs the album number: on Bandcamp choose Share / Embed, copy the embed code and paste it into the card
YouTube	Uses YouTube's privacy-enhanced player
Vimeo	Any `vimeo.com` link, unlisted ones included, as long as the video allows embedding
TikTok	The full `tiktok.com/@you/video/<id>` address (short `vm.tiktok.com` links can't be embedded)
Twitch	Your stream, or the latest broadcast when you're offline
Pinterest	A single pin. Boards and profiles become link cards
Pandora	A link card, since Pandora has no embeddable player

None of the players load anything from the provider until the visitor clicks (see Visitors).

Content

Card	What it shows
Rich text	Formatted text with headings, lists, quotes and links. At one row tall it becomes a single line
Section title	A full-width heading for grouping cards
Image	A photo, with an optional caption and link
Photo gallery	Up to 20 photos as a swipeable carousel or a grid (the grid shows up to 9)
Countdown	Days, hours, minutes and seconds to a date, then your own "it's here" text, with an optional button
File download	A PDF of up to 10 MB: a CV, a menu, a price list

Contact and bookings

Card	Notes
Contact form	Opens a form with name, email, an optional phone number and a message. Messages arrive in **More > Inbox**, and visitors never see your email address
Newsletter	An email box that signs people up with Buttondown, Mailchimp, Kit or MailerLite (connect it under Settings > Integrations). Trellis doesn't keep the list itself
Booking	Your Cal.com or Calendly page, shown inside the card when it's tall enough
Save my contact	A button that downloads a `.vcf` contact card. Everything in it is public
Map	An OpenStreetMap map with a pin. Type an address and press Find

Shop and tips

Trellis never takes payments. These cards link to payment pages you set up yourself with Stripe Payment Links, PayPal.me, Ko-fi, Gumroad, Etsy or similar.

- **Product:** a photo, name, price, short description, an optional badge ("New", "−20%") and a Buy button.
- **Tip jar:** up to six amount buttons, each with its own payment link.

Everything else

Links to most other well-known sites get the right icon automatically: Threads, YouTube channels, GitLab, Reddit, Telegram, Signal, Dribbble, Behance and plenty more. Anything else gets a globe icon and the site's name. You can also give a link card your own thumbnail picture.

Card settings

Every card has a **Card settings** section at the bottom of its dialog:

- **Visibility:** hide the card, or only show it between two dates. Hidden cards are never sent to visitors' browsers.
- **Lock:** show a sensitive-content warning or an age check first, or keep the card locked behind a code. A code-locked card's content stays on the server until someone types the right code (ten tries per 15 minutes). The warning and the age check are a courtesy; nothing is verified.
- **Look:** the card's own background and text colours, and a spotlight animation (pulse, wiggle, shine, bounce or glow) for the one card you want people to notice.

Instagram feed

Meta only lets you show your own posts through its official API, so this takes a few steps:

1. Make the Instagram account a Business or Creator account (it's free, in the Instagram app under Settings > Account type and tools).
2. At https://developers.facebook.com, create a Business app, add the Instagram product and choose *API setup with Instagram login*.
3. Under *Generate access tokens*, add the account and generate a token.
4. In the editor, go to **Settings > Integrations**, paste the token and press Connect.

The token stays on the server, and the server renews it every week or so, before Instagram's 60-day expiry.

Live data

Bluesky, Mastodon, Instagram, GitHub and blog cards are filled in by your server, which caches the data for ten minutes. Visitors' browsers never talk to those services directly.

If an embedded Notion page stays blank, check it's published to the web and that embedding isn't switched off in Notion. It still works as a link either way.

Logos belong to the companies they represent. See Design before recolouring them on a business site.

Design

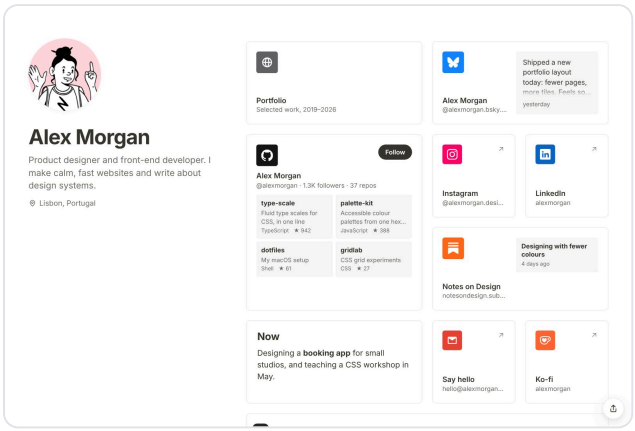
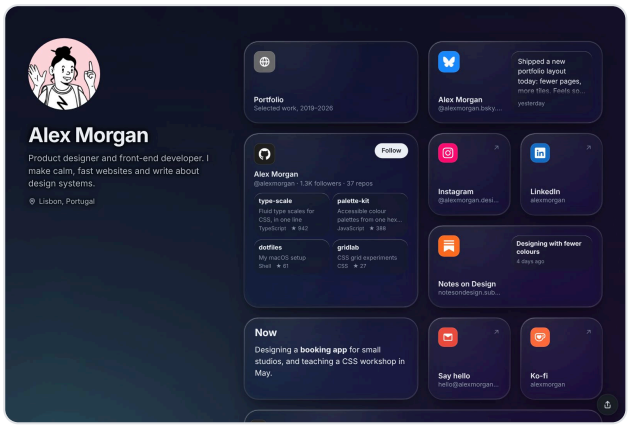
The palette button in the toolbar opens the Design panel. On a big screen it slides in from the side, and on a phone or tablet it comes up from the bottom, so the page stays visible either way. Changes show straight away and can be undone like anything else.

Presets

A preset sets the colours, background, card style, font and animation in one go. Treat it as a starting point and change whatever you like afterwards. You can also save your own designs under **My presets** (see Customising).

Preset	What it looks like
Default	Plain light or dark, following the visitor's device
Paper	Warm off-white, serif type, outlined cards
Mono	Very minimal: white page, black pill buttons, monochrome icons, a black-and-white drawn avatar
Notebook	White page, hairline cards, small corners, Inter
Notebook Dark	The same in dark grey
Sage	Soft green, rounded type, slow waves
Midnight	Navy gradient, glass cards, stars
Dusk	Pink-to-blue pastel gradient, glass cards, aurora
Glacier	Icy gradient, Crystal cards
Obsidian	Near-black with Crystal cards and flowing colour
Sunset	Peach-to-coral gradient, rounded type, waves
Terminal	Green on black, monospace, a grid and stars
Blossom	Pink, rounded type, floating orbs
Neon Grid	Synthwave purple and pink with the retro grid
Espresso	Coffee browns with a caramel accent and Playfair Display
Linen	Warm paper, terracotta, serif type, square corners
Ocean	Pale sea-blue gradient, glass cards, waves
Forest	Dark green, lime accent, fireflies
Winter	Night-blue gradient and falling snow
Mint	Mint-to-sky gradient, Crystal cards, aurora
Pop	Butter yellow, violet accent, pill buttons
Brutalist	Grey and black, thick outlines, no rounded corners, no animation

Noir	Black with a red accent and Playfair Display
Citrus	Cream and amber, dotted background, pill buttons

Notebook	Obsidian
	

Mode and colours

Mode is Auto (follow each visitor's light or dark setting), Light or Dark. The one exception is a gradient background that's clearly light or clearly dark: then the gradient wins, so you never get white text on a pale background. The panel tells you when that's happening.

The **accent** colour is used for buttons, links and the animated effects. You can also set your own background, card, text and secondary text colours, or reset any of them to follow the theme. Every colour has a box for a hex code if you'd rather type one.

Background

The background can be a plain colour, a gradient (two colours and an angle) or a photo. Photos can be darkened and blurred, and over a photo your profile sits on a frosted panel so it's still easy to read. Dots or a grid can go on top of any of them.

There are also nine animated backgrounds: aurora, flow, orbs, waves, stars, fireflies, snow, ripples and a retro grid. The retro grid is a neon floor with light cycles racing about. Click or tap the background and it sends out a shockwave, and you can draw on it by dragging. Each effect has a Slow, Normal and Fast speed and uses your accent colour. They stop moving for anyone who's asked their device to reduce motion.

Text that sits straight on an animated background or a photo (your name, bio and section titles) gets a faint halo, so it stays readable when something bright moves behind it.

Cards

Style	Look
Solid	White or dark cards with a soft shadow
Glass	Frosted and see-through

Crystal	Clear, glossy glass with a bright edge and a highlight that follows the mouse. Looks best over a gradient, photo or animated background
Outline	A crisp border and no shadow
Flat	Soft tinted blocks
Hairline	Plain cards with a very thin border

Glass and Crystal get a **Clarity** slider for how much of the background shows through. **Corner radius** goes from square (0) to very round (36).

Buttons

The Buttons section styles cards that are one row tall, which is how you get the classic stack of full-width links. Pick a shape (like the other cards, pill or square) and a fill (like the other cards, solid accent, outline or a soft tint). Text on a solid button turns black or white, whichever reads better.

If you don't have any one-row cards yet, the panel offers to turn your link cards into buttons in one step, and that's undoable too.

Fonts

System, Serif, Mono and Rounded use fonts that are already on the visitor's device. Inter, Space Grotesk, Outfit and Playfair Display are open-source web fonts that ship with Trellis and are served from your own server, never from Google Fonts. Visitors only download the one you're using.

Profile picture

Without a photo, Trellis draws an illustrated avatar from your name using DiceBear. The button under it picks a style (Sketch, Lorelei, Peeps, Cartoon, Thumbs, Robots, Pixel, Shapes, Glass, or plain initials), **Shuffle** draws a different one, and **Reset** goes back to the one based on your name. Avatars are drawn on your server, so nothing is sent to DiceBear, and only styles that are free for commercial use are included.

Design > Avatar adds an animation to the picture, photo or not: float, breathe, glow, pulse, a ring, wobble, spin or glitch. Auto, the default, gently moves drawn avatars and leaves photos alone. Black & white shows the picture in greyscale on a white circle. A few presets pick an animation for you, and Mono switches to the black-and-white Sketch avatar (it never replaces a photo you've uploaded).

Icons

Design > Icons changes how the platform logos on your page are coloured: in their brand colours (the default), tinted, in your accent colour, in your text colour, on a soft grey square, or as outlines. The "Follow" and "Subscribe" buttons on cards follow along.

The logos are trademarks of their owners, and Trellis only shows them to link to your profiles. Some brands ask that their logo only appears in its official colours, or in plain black or white, so check their guidelines before recolouring them on a business or client site.

Animation

None, Subtle (cards fade in one after another and lift a little on hover) or Lively (a gentle tilt on hover as well).

Credits

Brand icons come from Simple Icons, released under CC0, though the logos themselves still belong to their owners. Avatars are made with DiceBear (MIT), and the four web fonts are under the SIL Open Font License.

Customising

Most changes happen in the editor and need nothing else. If you run the server you can also install presets for every site on it, and if you're happy editing the code you can add fonts, effects, card styles and card types.

Presets

A preset is a saved design: mode, colours, background, effect, card style, radius, font, animation and so on. Applying one copies those values into the page, so you can carry on changing things afterwards, and Undo puts it all back. Presets never touch your cards, text or profile.

The Design panel shows the built-in presets, then any installed on the server, then the ones saved on this page under **My presets**.

Saving your own

Get the page looking the way you want, then under **My presets** click **Save current design** and give it a name. Saving again under the same name updates it. Hover a saved preset to export or delete it.

Saved presets are stored in the page itself, so they're in your backups and in the JSON export. A page can keep up to 24.

Moving a design to another site

Export downloads a small `.json` file and **Import** adds one to My presets on another site. Uploaded background photos aren't included, because they only exist on the server they were uploaded to, so upload the photo again on the other side. Background images given as an `https://` address are kept.

Offer presets to every site on a server

Put exported `.json` files straight into the `presets/` folder of the install, for example `/opt/trellis/presets/studio.json`, and they show up in every editor under "On this server". There's no need to restart. There are a couple of examples in `presets/examples/` to start from.

If you run several sites, point them all at one folder with `PRESETS_DIR=/etc/trellis/presets` in each site's `.env`. Broken files are skipped with a warning in the log. Up to 50 files are read, and anything in a sub-folder is ignored. The update command leaves `presets/` alone, so your files survive updates. With Docker, mount a folder at `/app/presets`.

The preset file format

```
{
  "trellisPreset": 1,
  "name": "Forest",
  "theme": "dark",
  "accent": "#4ade80",
  "appearance": {
    "colors": { "bg": "#0c1410", "card": "#14201a", "text": "#e7f3ea", "muted": "#8fa898" },
    "background": { "type": "gradient", "color1": "#0c1410", "color2": "#173326", "angle": 170,
"effect": "fireflies", "speed": "slow" },
    "cardStyle": "glass",
    "glassClarity": 60,
    "radius": 20,
    "font": "outfit",
    "animation": "subtle"
  }
}
```

Only `name` and `appearance` are required. Anything you leave out gets the default, and anything the server doesn't recognise is ignored. Colours are hex codes.

Field	Values	Default
`name`	Up to 40 characters	required
`theme`	`auto`, `light`, `dark`	`auto`
`accent`	Hex colour	`#0a84ff`
`appearance.colors.bg` / `card` / `text` / `muted`	Hex colour, or `""` to follow the theme	`""`
`appearance.background.type`	`none`, `gradient`, `image`	`none`
`appearance.background.color1` / `color2`	Gradient colours (also used by Aurora and Flow)	`#a1c4fd` / `#c2e9fb`
`appearance.background.angle`	0 to 360	`135`
`appearance.background.image`	An `https://` image address	`""`
`appearance.background.dim` / `blur`	Darken (0 to 80) and blur (0 to 30) a photo	`0`
`appearance.background.pattern`	`none`, `dots`, `grid`	`none`
`appearance.background.effect`	`none`, `aurora`, `flow`, `orbs`, `waves`, `stars`, `fireflies`, `snow`, `ripples`, `grid`	`none`
`appearance.background.speed`	`slow`, `normal`, `fast`	`normal`
`appearance.cardStyle`	`solid`, `glass`, `crystal`, `outline`, `flat`, `hairline`	`solid`
`appearance.glassClarity`	0 (frosted) to 100 (clear)	`50`
`appearance.radius`	0 to 36	`24`

`appearance.font`	`system`, `serif`, `mono`, `rounded`, `inter`, `grotesk`, `outfit`, `playfair`	`system`
`appearance.animation`	`none`, `subtle`, `lively`	`subtle`
`appearance.buttonShape`	`card`, `pill`, `square`	`card`
`appearance.buttonFill`	`card`, `solid`, `outline`, `soft`	`card`
`appearance.iconStyle`	`brand`, `tinted`, `accent`, `mono`, `soft`, `outline`	`brand`
`appearance.avatarTone`	`color`, `mono`	`color`
`appearance.avatarEffect`	`auto`, `none`, `float`, `breathe`, `glow`, `pulse`, `ring`, `wobble`, `spin`, `glitch`	`auto`

Changing the code

The rest of this page assumes you're comfortable with the code. Whatever you add, the client and the server both need to know about it, because the server rejects values it doesn't recognise. Run `npm run lint` and `npm test` afterwards. Development explains how the code is laid out.

Built-in presets

They're the `PRESETS` list in `client/src/lib/appearance.js`, in the same shape as a preset file plus a `key`. Pages that used a preset you remove keep their design, because applying a preset copies it into the page. For a design only your own sites need, a server preset file is usually less work.

A new font

1. `npm i -w client @fontsource-variable/<font>` and import its CSS in `client/src/main.jsx`. Don't link Google Fonts: the Content Security Policy blocks them, and self-hosting keeps visitors' IP addresses private.
2. Add it to `FONT_STACKS` in both `client/src/lib/appearance.js` and `server/src/lib/appearance.js`, and to the `font` list in `server/src/lib/validate.js`.
3. Add a button for it in `DesignPanel.jsx` and a preview rule in `editor.css`.

A new animated background

Add the key to the `effect` list in `validate.js`, render it in `PageBackground.jsx`, and style it as `.fx-<key>` in `base.css`. Stick to animating `transform` and `opacity` so it stays smooth on phones, and multiply durations by `var(--fx-speed)`. Then add it to the list in `DesignPanel.jsx`. Visitors who prefer reduced motion get it switched off automatically.

A new card style

Add it to the `cardStyle` list in `validate.js`, style it with `:root[data-cards='<key>'] .tile-cell` in `base.css`, then add it to the panel and to the preview in `PresetPicker.jsx`.

Default colours and the starter page

The light and dark palettes live at the top of `client/src/styles/base.css`. If you change them, update `THEME_DEFAULTS` in `appearance.js` to match. New installs start from `server/src/lib/defaultPage.js`.

For new card types and platform icons, see adding a card type and adding a platform icon.

Keeping changes across updates

Anything you do in the editor, presets included, is part of the page data and updates never touch it. Server preset files survive updates too. Code changes don't: a new release replaces the code, so keep your changes on a branch or a fork, merge new versions into it and build your own zip.

Site name, favicon and sharing

These all live under **Settings** (the gear button).

Site name

The site name is what appears in the browser tab and bookmarks, in search results, in link previews on WhatsApp, iMessage, Slack and the like, and under the icon when someone adds the page to their phone's home screen. The settings show a little preview of the tab and the link preview as you type. Leave it empty to use the name from your profile.

Description

Shown in search results and link previews. If you leave it empty, the start of your bio is used.

Favicon

The default is the Trellis icon. You can pick an emoji instead (it's turned into a crisp SVG icon) or upload a square image; a 512 × 512 PNG is ideal. An uploaded image is also used as the home-screen icon.

Link previews

When someone shares your page, chat apps and social networks show a card with the site name, description and your profile photo. **Settings > Link preview image** lets you choose a different picture, cropped to the 1200 × 630 those apps expect.

If the page is hidden from crawlers (see Search engines and AI), link previews are blocked as well, unless you tick **Still allow link previews in chat apps**.

The "Powered by Trellis" footer

There's a small "Powered by Trellis" link at the bottom of the page. Untick **Settings > Footer > Show "Powered by Trellis"** to remove it. It's set per page, so you can keep it on your own and turn it off on a client's.

Visitors, forms and statistics

All of this runs on your own server. Nothing goes to Google or an analytics company, and visitors don't get any cookies, so there's no need for a cookie banner.

Statistics

More > Statistics shows the last 7, 30 or 90 days, or the whole year: views, unique visitors, clicks and the click rate, a chart per day, your most clicked cards, short link visits, the sites people came from, countries (or browser languages), devices, browsers, operating systems and campaign tags. **Export CSV** downloads the lot.

The page sends a tiny beacon when it opens and when someone clicks a card. Bots, AI agents, you (while you're signed in) and anyone whose browser sends *Do Not Track* or *Global Privacy Control* aren't counted.

Unique visitors are worked out the way Plausible and Fathom do it: a hash of the IP address and browser with a random key that changes every day and is never saved. No IP addresses are stored, only daily totals, and days older than 400 are deleted. Days run midnight to midnight UTC.

You can switch statistics off under **Settings > Statistics**.

Campaigns

Share the page as `https://yoursite.com/?utm_source=instagram&utm_campaign=spring` and those names show up under Campaigns (`?ref=newsletter` works as well).

Tag outgoing links, in the same settings, does the opposite: links to other sites get `utm_source=<your site>&utm_medium=link-in-bio` added, so their statistics show the visit came from you. Links that already have `utm_` tags are left alone.

Countries

Your server only knows the visitor's country if the proxy in front of it says so. Cloudflare sends a `CF-IPCountry` header, so behind Cloudflare it just works. With nginx you can use the GeoIP2 module and the free MaxMind GeoLite2 database:

```
geoip2 /usr/share/GeoIP/GeoLite2-Country.mmdb {
    $geoip2_country_code country iso_code;
}
# in the location block that proxies to Trellis:
proxy_set_header X-Country-Code $geoip2_country_code;
```

and then set `GEO_HEADER=X-Country-Code` in `.env`. Without a country header, the statistics show browser languages instead.

If you're not behind Cloudflare, it's worth clearing that header in nginx (`proxy_set_header CF-IPCountry "";`), otherwise anyone could send a made-up country.

Behind Cloudflare

Install `deploy/cloudflare-real-ip.conf` as described in Deployment. Without it, everyone reaching you through the same Cloudflare data centre looks like a single visitor.

Click-to-load players

YouTube, Vimeo, Spotify, SoundCloud, TikTok, Instagram, maps, booking pages and the other embeds start out as a small notice with a **Load** button. Nothing is fetched from those companies until the visitor clicks, which is what EU privacy rules expect. They can tick "Always load from YouTube" (or whichever service it is), and that choice is remembered in their own browser.

If you'd rather players load straight away, switch it off under **Settings > Visitors**.

Contact form and inbox

The Contact form card opens a form asking for a name, an email address, an optional phone number and a message. Messages land in **More > Inbox**, where you can reply by email, mark them unread, delete them or export them as CSV.

Each visitor can send five messages every ten minutes (contact and newsletter forms together, and a message rejected for a typo doesn't count). Known bots are turned away, and a hidden field that only bots fill in catches most of the rest.

Getting an email for each message

Give the server an SMTP account in `.env`, either your own email provider's or a sending service like Mailgun, Postmark, Brevo or Amazon SES:

```
SMTP_URL=smtps://user:password@smtp.example.com:465
NOTIFY_EMAIL=you@example.com
SMTP_FROM=Trellis <noreply@example.com>   # optional
```

Restart the server afterwards. Replying to the email goes straight to the visitor. Messages are always saved to the inbox first, so a failed email doesn't lose anything. Special characters in the password need URL-encoding (`@` becomes `%40`).

Messages are personal data, so delete the ones you don't need and mention the form in your privacy notice if you have one.

Newsletter sign-ups

Connect your newsletter under **Settings > Integrations**, then add a Newsletter card.

Service	What you need
Buttondown	An API key
Mailchimp	An API key and the audience ID

Kit (ConvertKit)	A v4 API key and the form ID
MailerLite	An API token, and optionally a group ID

Email addresses go straight to the service; Trellis doesn't keep a list. Mailchimp always sends a confirmation email. For the others, turn on double opt-in in the service itself if you want it.

Share button

The round button in the corner opens a share sheet with a QR code, Copy link, the phone's own share menu, and the QR code as a PNG or SVG download for print. It can be turned off under **Settings > Visitors**.

Search engines and AI

There are two switches under **Settings > Privacy**. They take effect when you publish, and you don't need to change anything in nginx.

Hide from search engines and all crawlers

This keeps the page out of Google and every other search engine, and turns away SEO tools, web archives, AI crawlers, link-preview fetchers, scripts like `curl` or Python, headless browsers, and anything else that identifies itself as a bot. People using a normal browser don't notice a thing.

Here's how it works:

- Every response, images included, carries a `noindex` header, and the page also has a `noindex` meta tag.
- Anything recognised as a crawler gets `403 Forbidden` instead of the page, so even crawlers that ignore robots.txt get nothing.
- `robots.txt` tells AI bots, SEO tools and archives to stay away.

You might notice that robots.txt doesn't tell search engines to stay away, and that's deliberate. If it did, Google would never fetch the page, would never see the `noindex`, and could still list your bare address when other sites link to it (Google says as much). Instead, search engines are allowed to ask and get the 403, and Google and Bing don't index pages that answer 403 (and drop them if they had them).

If the page is already in search results, it disappears as the engines visit again, usually within a few days or weeks. Google Search Console's Removals tool is quicker.

Hiding the page also blocks the previews that WhatsApp, iMessage and Slack show when someone shares your link, because those come from bots too. Tick **Still allow link previews in chat apps** to let them through. It also breaks Mastodon's link verification (the green tick), unless you allow previews.

Block AI crawlers and agents

This one leaves search engines alone and blocks AI: the bots that collect training data, AI search, and assistants and agents that browse on someone's behalf. That covers OpenAI, Anthropic, Google's AI crawlers and agents (Gemini, NotebookLM), Apple's AI training, Amazon, Meta, Microsoft, Perplexity, Mistral, DeepSeek, Common Crawl, ByteDance and about a hundred others. The list is `AI_BOTS` in `server/src/lib/robots.js`, and it's based on the community ai.robots.txt list.

Some AI agents, such as ChatGPT agent, browse with an ordinary browser and don't name themselves, but the reputable ones sign their requests (Web Bot Auth). Signed requests are turned away as well.

Hiding the page from all crawlers includes all of this.

The editor

Whatever these switches are set to, the editor is never shown to a bot, is never indexed or cached, and can't be framed by another site. Its address is not in robots.txt either, because robots.txt is public.

Limits

The big search engines and AI companies respect all of this. No website can stop a scraper that pretends to be a normal browser, though, so if a page is genuinely private, put a password in front of it in nginx.

Short links (`yoursite.com/cv`) still forward everyone, bots included, even when the page is hidden. A redirect only gives away where it points, and it means link previews of a short link show the destination.

Checking it

```
curl -I -A "Mozilla/5.0 (compatible; Googlebot/2.1)" https://your-domain/   # 403 when hidden
curl -A "Mozilla/5.0" https://your-domain/robots.txt
```

With the page hidden, plain `curl` is refused too, so give it a browser-like User-Agent with `-A` when you want to look at the page.

Deployment

In production Trellis is a single Node.js process behind nginx, with nginx handling HTTPS (Let's Encrypt is fine). These steps assume a Linux server with systemd. There's a Docker section further down if you'd rather use that.

1. Build the release zip

On your own computer:

```
npm ci
npm run build
```

That builds the front end and writes `release/trellis-<version>.zip`, plus a `.sha256` checksum next to it. The zip contains the server, the built front end and the production dependencies, so the server doesn't need to run `npm install` or build anything. It never includes your data or `.env` files.

2. Install it

The server needs Node.js 22.9 or newer and `unzip`.

```
scp release/trellis-1.0.0.zip you@yourserver:/tmp/

# then, on the server:
cd /tmp && unzip -q trellis-1.0.0.zip
sudo mv trellis-1.0.0 /opt/trellis
cd /opt/trellis
sudo cp .env.example .env
sudo sh deploy/fix-permissions.sh
sudo -u www-data npm run set-password
sudo -u www-data npm run admin-url
```

In `.env`, set `PUBLIC_URL` to the site's address (`https://example.com`), and leave `TRUST_PROXY=1` and `HOST=127.0.0.1` as they are. The server warns you at start-up if `PUBLIC_URL` is missing.

`fix-permissions.sh` makes root the owner of the code and gives the app's user (`www-data`) only `server/data`, so the app can never change its own code. Run it again after updating by hand. The update command does it for you.

3. systemd

```
sudo cp deploy/trellis.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now trellis
journalctl -u trellis -f
```

The unit runs the app as `www-data` in production mode, only lets it write to `server/data`, and locks down the rest with systemd's sandboxing options. Change `User` or `WorkingDirectory` if your setup is different.

4. nginx

`deploy/nginx.conf` is a complete example: a redirect from HTTP to HTTPS, the HTTPS site proxying to `127.0.0.1:3001` with a rate limit on sign-ins, and a catch-all server that drops requests for host names you don't serve. You can only have one catch-all per port, so delete that block if you already have one.

The proxy headers are in `deploy/trellis-proxy.conf`. It overwrites the `X-Forwarded-*` headers rather than passing on whatever the visitor sent.

```
sudo cp deploy/trellis-proxy.conf /etc/nginx/snippets/
sudo cp deploy/nginx.conf /etc/nginx/sites-available/trellis
# set server_name and the certificate paths, then:
sudo ln -s /etc/nginx/sites-available/trellis /etc/nginx/sites-enabled/
sudo certbot --nginx -d example.com -d www.example.com   # if you need a certificate
sudo nginx -t && sudo systemctl reload nginx
```

Adding it to an existing site? Copy over the two `location` blocks, the `limit_req_zone` line and `client_max_body_size 11m;`. Keep the `X-Forwarded-Proto $scheme` header, because that's how the app knows a request came in over HTTPS.

Behind Cloudflare

With Cloudflare in front, nginx sees Cloudflare's IP address instead of the visitor's. Everyone coming through the same Cloudflare data centre then looks like one person: unique visitors are undercounted, and the limits on sign-ins and the contact form are shared between them. The fix is a small file that tells nginx to trust Cloudflare's `CF-Connecting-IP` header, but only on requests that really come from Cloudflare:

```
sudo cp deploy/cloudflare-real-ip.conf /etc/nginx/conf.d/
sudo nginx -t && sudo systemctl reload nginx
```

The file lists Cloudflare's IP ranges, along with a one-liner to refresh them if they ever change.

5. Once it's running

Check the page loads and that a made-up address like `/test` shows the 404 page. Sign in, turn on two-step sign-in (More > Sign-in & security) and keep the recovery codes safe. Then press **Settings > Backup > Back up now** to make sure backups work, and think about an off-site copy.

A quick checklist:

- `http://` redirects to `https://`, and `curl -I https://your-domain/` shows a `Strict-Transport-Security` header.
- `/admin` is a 404 and your editor address shows the sign-in page.
- If signing in silently does nothing, check `TRUST_PROXY=1` and the `X-Forwarded-Proto` header.
- Statistics count a visit from your phone when you're not signed in.

HTTPS-only mode

In production the app insists on HTTPS. It redirects `http://` requests, sends HSTS (without `includeSubDomains`, so your other subdomains aren't affected), and its Content Security Policy only lets the page load scripts from itself and frames from the embed providers it knows. Image and avatar addresses are always saved as `https://`, and the sign-in cookie is `Secure`, `HttpOnly` and `SameSite=Strict`. Requests to `localhost` are exempt, so local development keeps working.

This needs `TRUST_PROXY=1` behind nginx. Without it the app can't tell a request arrived over HTTPS, so it skips its own redirect and logs a warning. The systemd units already set it. `FORCE_HTTPS=0` turns the whole thing off.

Docker

```
docker build -t trellis .
docker run -d --name trellis --restart unless-stopped \
  -p 127.0.0.1:3001:3001 -v trellis-data:/data \
  -e NODE_ENV=production -e TRUST_PROXY=1 trellis
docker exec -it trellis node server/scripts/set-password.js
docker exec trellis node server/scripts/admin-url.js
```

Put the same nginx config in front of it. `deploy/docker-compose.yml` shows how to run several sites. To update, rebuild the image and recreate the container; the data volume stays.

Updating

Build the new zip, copy it over and run:

```
cd /opt/trellis
sudo npm run update -- /tmp/trellis-1.0.0.zip
```

The update backs up the data folder first, swaps in the new code (keeping your data, `.env` and any server presets), restarts the service and waits for it to come back. If the new version hasn't started properly within 45 seconds, it puts the old one back and tells you to look at `journalctl -u trellis -n 50`.

The previous version stays in `/opt/trellis.previous`, so going back is one command:

```
sudo npm run update -- --rollback
```

`sudo npm run update -- --help` lists the options. The ones you're most likely to need are `--force` to reinstall the same or an older version, `--yes` to skip the question, and `--service` with `--health` when you run several sites from one install, for example `--service trellis@alice --health http://127.0.0.1:3101/api/health`.

From a pre-release build

Servers running one of the test builds from before 1.0 (versions 1.2 to 1.4.6) see 1.0.0 as older and stop, so add `--force` the first time:

```
sudo npm run update -- --force /tmp/trellis-1.0.0.zip
```

Builds older than 1.2 don't have the update command yet. Run the one inside the new zip instead:

```
cd /tmp && unzip -q -o trellis-1.0.0.zip
sudo node /tmp/trellis-1.0.0/server/scripts/update.js --dir /opt/trellis --force /tmp/trellis-1.0.0.zip
```

Pages from older versions are converted automatically.

▶ **Updating by hand**

Configuration

Server settings come from environment variables, normally in a `.env` file in the project folder (start from `.env.example`). All of them are optional. Everything to do with the page itself, its design and its privacy settings is done in the editor.

Environment variables

Variable	Default	What it's for
`PORT`	`3001`	The port the app listens on
`HOST`	`127.0.0.1`	The address it binds to. Keep `127.0.0.1` behind nginx on the same machine; Docker uses `0.0.0.0`
`NODE_ENV`		`production` on a server, which turns on HTTPS-only mode
`PUBLIC_URL`		The site's address, e.g. `https://example.com`. Redirects and link previews use it instead of trusting the request. Set it in production
`TRUST_PROXY`		`1` behind nginx, Caddy or Traefik, so the app sees HTTPS and real client IPs
`FORCE_HTTPS`	on in production	`0` turns HTTPS-only mode off
`DATA_DIR`	`server/data`	Where the page, uploads, password hash and keys live
`ADMIN_PATH`	random	Choose the editor's secret path yourself (16 to 128 letters, digits, `-` and `_`). Usually best left alone
`SESSION_SECRET`	generated	Key for signing the sign-in cookie. Generated into `DATA_DIR/secret.key` if not set
`ADMIN_PASSWORD`		A password to set on first start. `npm run set-password` is better; remove this once it's done
`TRELLIS_ENV_FILE`	`.env`	Read settings from a different file, or `none` to skip it. Real environment variables always win
`PRESETS_DIR`	`presets/`	Folder of server presets (see Customising)
`INSTAGRAM_ACCESS_TOKEN`		An alternative to connecting Instagram in the editor. It isn't renewed automatically
`SMTP_URL`		Mail server for contact form emails, e.g. `smtps://user:password@smtp.example.com:465`
`NOTIFY_EMAIL`		Where those emails go
`SMTP_FROM`	`NOTIFY_EMAIL`	The sender address
`BACKUP`	on	`off` stops the nightly backups (`npm run backup` still works)
`BACKUP_TIME`	`03:30`	When the nightly backup runs, in server time
`BACKUP_KEEP`	`14`	How many backups to keep
`BACKUP_DIR`	`DATA_DIR/backups`	Where backups go

`BACKUP_COPY_DIR`		A second folder for copies, such as a mounted NAS. Under systemd, add it to `ReadWritePaths`
`BACKUP_S3_BUCKET`		Upload each backup to S3-compatible storage, along with `BACKUP_S3_ENDPOINT`, `_REGION`, `_PREFIX`, `_ACCESS_KEY` and `_SECRET_KEY`
`GEO_HEADER`	`CF-IPCountry`	The header that carries the visitor's country (see Visitors)

The command-line scripts read the same `.env`, so they always work on the right data folder.

Commands

Command	What it does
`npm run dev`	Development: API on port 3001, Vite with hot reload on 5173
`npm run build`	Build the front end and make the release zip
`npm run build:client`	Build the front end only
`npm start`	Run the app, serving the built front end
`npm run set-password`	Set or change the editor password
`npm run admin-url`	Show the editor address. `-- --rotate` replaces it
`npm run backup`	Back up now. `-- --list` lists the backups
`npm run restore -- latest`	Restore the newest backup, or `-- <file>` a particular one. Stop the server first
`npm run update -- <zip>`	Install a new release. `-- --rollback` goes back
`npm run reset-2fa`	Turn off two-step sign-in if you've lost your phone and recovery codes
`npm run sign-out-all`	Sign every device out of the editor
`npm run lint`	Lint the front end
`npm test`	Build the front end and run all the tests

Addresses

Path	What's there
`/`	The page
your secret path	The editor
`/robots.txt`	Generated from the privacy settings
`/uploads/…`	Uploaded images and files
`/api/…`	The JSON API (see Development)

`/api/health`	Returns `{ "ok": true }` for uptime monitors. Requests from the server itself also get the version number
`/cv`, `/github` …	Your short links
anything else	The 404 page, `/admin` included

The page doesn't take any query parameters, so `/?page=2` gets the 404 page too. The exceptions are tracking parameters that social networks and newsletters add to shared links (`?fbclid=`, `?igsh=`, `?utm_source=` and so on), which keep working. The list is in `server/src/lib/params.js` if a service you use adds one that's missing.

Data and backups

Where everything is

There's no database. Everything for a site is in one folder, `server/data/` unless `DATA_DIR` says otherwise:

File	What's in it
`page.json`	The published page: profile, design, settings and cards
`draft.json`	Unpublished changes, while there are any
`versions/`	Version history
`uploads/`	Uploaded photos and PDFs
`analytics.json`	Statistics (daily totals only, no IP addresses)
`inbox.json`	Contact form messages
`shortlinks.json`	Short links and their visit counts
`signins.json`	The last 30 sign-in attempts
`auth.json`	The password, as a scrypt hash
`twofactor.json`	The two-step sign-in secret, if it's on
`secret.key` , `sessions.json` , `admin-path`	The cookie key, who's signed in, and the editor address
`integrations.json`	Instagram and newsletter tokens, if connected
`backups/`	The automatic backups

The files with secrets in them are only readable by the app's own user, and the whole folder is in `.gitignore` . Inbox messages and sign-in attempts are personal data, so treat backups with the same care.

Backups

Every night at 03:30, server time, the app packs the whole data folder into a `.tar.gz` in `server/data/backups/` and keeps the newest 14. If the server was off at that time, it catches up a minute after it starts again. **Settings > Backup** in the editor shows when the last one ran.

```
sudo -u www-data npm run backup            # back up now
sudo -u www-data npm run backup -- --list  # see what's there
```

`BACKUP_TIME` , `BACKUP_KEEP` and `BACKUP_DIR` in `.env` change when, how many and where. `BACKUP=off` stops the nightly run.

Off-site copies

A backup on the same disk won't help if the disk dies, so it's worth sending a copy somewhere else. The simplest option is a second folder, such as a NAS share mounted on the server:

```
BACKUP_COPY_DIR=/mnt/nas/trellis-backups
```

Under systemd the app isn't allowed to write there until you add the folder to `ReadWritePaths` (`sudo systemctl edit trellis`), for example `ReadWritePaths=/opt/trellis/server/data /mnt/nas/trellis-backups`.

You can also upload to S3-compatible storage (AWS S3, Cloudflare R2, Backblaze B2, Wasabi, MinIO):

```
BACKUP_S3_ENDPOINT=https://s3.eu-central-1.amazonaws.com
BACKUP_S3_REGION=eu-central-1     # "auto" for R2
BACKUP_S3_BUCKET=my-backups
BACKUP_S3_PREFIX=trellis
BACKUP_S3_ACCESS_KEY=...
BACKUP_S3_SECRET_KEY=...
```

Give that key permission to upload and nothing else (`s3:PutObject`), so someone who steals it can't read or delete your backups. Trellis doesn't delete old backups from the bucket, so add a lifecycle rule there (delete after 30 days, say). If an upload fails, the local backup is still made and the error shows up in the editor.

Restoring

```
sudo systemctl stop trellis
sudo -u www-data npm run restore -- latest    # or a file name from --list
sudo systemctl start trellis
```

It shows you what's in the backup and asks before changing anything. The current data is backed up first, so you can undo a restore the same way. Everyone has to sign in again afterwards.

The backups are ordinary tar files, so you can also open them by hand (`tar -tzf trellis-....tar.gz`). To bring back only the page, use **Import JSON** in the editor with a file from **Export JSON**.

Moving a site

Install Trellis on the new server, copy the data folder over and start it. The password, editor address, design and content all come with it.

Resetting things

- New password: `npm run set-password` (this also signs everyone out).
- Sign out every device: More > Sign-in & security in the editor, or `npm run sign-out-all` on the server.
- Turn off two-step sign-in: `npm run reset-2fa`.
- A new editor address: `npm run admin-url -- --rotate`.
- Start the page from scratch: stop the server, delete `page.json` and start it again.

Multiple sites

One running copy of Trellis serves one page. To host pages for several people, run one instance per site from the same code, each with its own data folder, port and editor address. The sites can't see or affect each other, and each one can be backed up, moved or removed on its own.

With systemd

`deploy/trellis@.service` is a template unit that runs as many instances as you like from `/opt/trellis`. Each one gets a settings file in `/etc/trellis/<name>.env` and its data in `/var/lib/trellis/<name>`.

```
sudo cp deploy/trellis@.service /etc/systemd/system/
sudo mkdir -p /etc/trellis

# a site called "alice" on port 3101
printf 'PORT=3101\nHOST=127.0.0.1\nPUBLIC_URL=https://alice.example.com\n' | sudo tee
/etc/trellis/alice.env
sudo systemctl daemon-reload
sudo systemctl enable --now trellis@alice

# her password and editor address
cd /opt/trellis
sudo -u www-data env DATA_DIR=/var/lib/trellis/alice node server/scripts/set-password.js
sudo -u www-data env DATA_DIR=/var/lib/trellis/alice node server/scripts/admin-url.js
```

With Docker Compose

`deploy/docker-compose.yml` has two example sites built from one image, each with its own volume and port. Copy a service block to add another.

```
docker compose -f deploy/docker-compose.yml up -d --build
docker compose -f deploy/docker-compose.yml exec alice node server/scripts/set-password.js
docker compose -f deploy/docker-compose.yml exec alice node server/scripts/admin-url.js
```

nginx

Give each site its own domain or subdomain and certificate:

```
server {
    listen 443 ssl;
    http2 on;
    server_name alice.example.com;
    ssl_certificate     /etc/letsencrypt/live/alice.example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/alice.example.com/privkey.pem;
    client_max_body_size 11m;

    location / {
        proxy_pass http://127.0.0.1:3101;
        include snippets/trellis-proxy.conf;
    }
}
```

`sudo certbot --nginx -d alice.example.com` gets the certificate. Copy the port 80 redirect from `deploy/nginx.conf`.

Presets for everyone

Presets in `/opt/trellis/presets/` show up in every site's editor under "On this server". To give a site its own set, point it somewhere else with `PRESETS_DIR` in its `.env`. See Customising.

Handing a site over

Send the person their editor address and, separately, their password. Or set a temporary password and change it to one they choose. Point them at the editor guide and ask them to turn on two-step sign-in. If they ever lose their phone and their recovery codes:

```
sudo -u www-data env DATA_DIR=/var/lib/trellis/alice node server/scripts/reset-2fa.js
```

Backups and updates

Each site backs itself up every night into its own data folder. Add `BACKUP_COPY_DIR` or the S3 settings to a site's `.env` for an off-site copy.

One update covers every site, as long as you name each service and its health check:

```
cd /opt/trellis
sudo npm run update -- /tmp/trellis-<version>.zip \
  --service trellis@alice --health http://127.0.0.1:3101/api/health \
  --service trellis@bob   --health http://127.0.0.1:3102/api/health
```

If any of them doesn't come back up, they all go back to the previous version.

Each instance uses roughly 70 to 90 MB of memory and almost no CPU when it's idle, so a small VPS with 1 or 2 GB of RAM can run a dozen or so.

Removing a site

Take a last backup first, then:

```
sudo systemctl disable --now trellis@alice
sudo rm /etc/trellis/alice.env
sudo rm -r /var/lib/trellis/alice
```

Security

What Trellis does to protect a site, what's left up to you, and the things it can't do.

Signing in

The editor lives at a random path of about 144 bits, so it can't be guessed or found by scanning. It isn't linked anywhere or listed in robots.txt, is never shown to bots, and doesn't leak through the `Referer` header. Sign-in requests have to include that path as well as the password, so a bot that doesn't know the address can't even start guessing.

Passwords are at least eight characters, and they're stored as scrypt hashes and compared in constant time. The server allows ten sign-in attempts per IP address (or IPv6 /64) every 15 minutes, and 30 per IPv6 /48, so hopping between addresses doesn't get anyone further. The example nginx config adds a second limit in front of that.

Sessions live on the server. The browser just holds a signed cookie that's `HttpOnly`, `SameSite=Strict` and, over HTTPS, `Secure`, and that lasts 30 days. Logging out ends the session on the server, and changing the password signs out every device. Every change the editor sends also has to carry a custom header, which a form on another site can't add.

Two-step sign-in uses ordinary authenticator app codes (TOTP). Each code works once, recovery codes are stored hashed, and you need the password to turn it off or make new recovery codes, so someone who has only stolen a session can't switch it off. The editor also lists signed-in devices and recent sign-in attempts.

What gets saved

The server checks every save again from scratch. It drops unknown card types and fields, limits the length of text, only allows hex colours, and only allows `http(s)`, `mailto:` and `tel:` links. Rich text is cut down to a short list of tags, and the browser rebuilds it from that list rather than inserting HTML.

Uploads are identified by their contents rather than their name. JPEG, PNG, GIF, WebP, AVIF, ICO and PDF are accepted, but SVG isn't, because it can contain scripts. Location and camera details are removed from JPEG, PNG and WebP photos. PDFs are always downloaded rather than opened inside the site. Imported pages, backups and preset files go through the same checks.

In the browser

The Content Security Policy only lets the page run its own scripts (the inlined page data uses a nonce), and only lets it frame the embed providers Trellis knows about. Embeds are built from IDs parsed out of the link, never from the link itself. The usual headers are on every response: `X-Content-Type-Options`, `Referrer-Policy`, `Permissions-Policy` and `X-Frame-Options`.

In production it's HTTPS only, with HSTS. See Deployment.

Requests the server makes

Live cards (Bluesky, Mastodon, GitHub, Instagram and blogs) are fetched by the server, not the visitor. The feed endpoint takes a card ID rather than a URL, so it can't be used as an open proxy. Those requests are https only, on the normal port, with a size and time limit. They refuse private and internal addresses (checked after DNS resolution, and again on every redirect, so DNS tricks don't get round it). Results are cached for ten minutes.

Up to you

- Run it behind HTTPS with `TRUST_PROXY=1` and `PUBLIC_URL` set.
- Use a strong password for each site, and turn on two-step sign-in.
- Keep the editor address to yourself, and change it with `npm run admin-url -- --rotate` if it might have leaked.
- Keep a backup somewhere other than the server. Backups contain the password hash and keys, so keep them private.
- Keep Node.js and the server's operating system up to date.
- Behind Cloudflare, install `deploy/cloudflare-real-ip.conf` so the sign-in limits apply per visitor (see Deployment).

Things it doesn't do

- Two-step sign-in is optional. Authenticator codes don't protect against a convincing fake sign-in page that passes the code on in real time. Passkeys aren't supported yet.
- The app's own sign-in limit is kept in memory and resets when it restarts. The nginx limit doesn't.
- AVIF and GIF uploads are stored as they are. AVIF photos straight from some cameras may still include location data.
- If you switch off click-to-load, players load the provider's scripts as soon as the page opens, which may need consent in the EU.
- Bot blocking relies on bots telling the truth about who they are. A scraper pretending to be a normal browser can't be spotted. If a page has to stay private, put a password on it in nginx.
- Uploads that are no longer used aren't cleaned up automatically.
- All sites on one server run as the same user unless you set each one up with its own (see the comment in `deploy/trellis@.service`).

Reporting a problem

If you find a security problem, please don't open a public issue. Get in touch with me privately instead.

Troubleshooting

"Client not built yet" Run `npm run build` and restart the server.

The page is blank or half-styled after an update Restart the server (`sudo systemctl restart trellis`). It reads the HTML template when it starts, so it's still pointing at the old file names.

`node: bad option` or a syntax error at start-up Node.js is too old. You need 22.9 or newer.

I've lost the editor address Run `npm run admin-url` on the server, as the app's user and from the project folder. It's also in the log the very first time the server starts.

`/admin` says the page doesn't exist That's on purpose. The editor is at its own secret address.

A shared link opens the 404 page The link has a query parameter (the part after `?`) that the page doesn't know. The usual tracking parameters are allowed. Add others to `server/src/lib/params.js` and restart.

A short link opens the 404 page Short links have to match exactly (`/cv`, not `/cv.pdf`), although case doesn't matter. Check the name under More > Short links.

"Wrong password" when it isn't Sign-ins only work from the current editor address. After `admin-url -- --rotate` the old one stops working. After ten wrong attempts you have to wait 15 minutes.

Signing in does nothing, or signs me straight back out Over HTTPS the cookie is marked Secure, so the app has to know the request was HTTPS. Check `TRUST_PROXY=1` is set and nginx sends `proxy_set_header X-Forwarded-Proto $scheme;`.

Redirect loop Same cause: nginx has to send `X-Forwarded-Proto` and the app needs `TRUST_PROXY=1`. As a last resort, `FORCE_HTTPS=0` switches off the app's redirect and leaves nginx's in place.

I forgot the password `npm run set-password` on the server. It signs out every device as well.

I lost my phone and my recovery codes `sudo -u www-data npm run reset-2fa` turns two-step sign-in off. Sign in with your password and turn it back on.

"That code didn't work" with the right code Authenticator codes depend on the time, so check the clocks on your phone and the server are set automatically (`timedatectl` on the server). A code only works once, so wait for the next one.

"Please sign in again" while editing The session ran out (they last 30 days) or the password was changed. Your changes are still there; sign in and they get saved.

"Edited elsewhere" The page was saved from another tab or device. The editor asks which version to keep; see two tabs or two devices.

`npm run update` rolled back The new version didn't pass its health check within 45 seconds, so the old one is running again and the failed copy is in `/opt/trellis.failed`. `journalctl -u trellis -n 50` shows why. It's often the Node.js version or a typo in `.env`.

The off-site backup copy failed For a folder, it has to be mounted and writable by the app's user, and listed in `ReadWritePaths` under systemd. For S3, check the endpoint, region and bucket, and that the key is allowed to `PutObject`. The editor shows the error the storage service gave.

A blog card has no posts Latest posts work for public Medium, Substack, WordPress.com, Blogger and Tumblr blogs. Look for `[feeds]` lines in the server log.

Bluesky, Mastodon or GitHub cards show nothing The server has to be able to reach `public.api.bsky.app`, your Mastodon server or `api.github.com`. Results are cached for ten minutes, and GitHub allows 60 requests an hour per server without a token. Check the log for `[feeds]` lines.

The Instagram feed is empty Connect Instagram under Settings > Integrations and reload the editor. The account has to be a Business or Creator account. If the token was revoked, connect it again.

An embed is just a grey box Check it's a proper share link for a playlist, album, song or video. Some networks block those services entirely.

Uploads fail Photos can be JPEG, PNG, GIF, WebP, AVIF or ICO, and file cards take PDFs up to 10 MB. A "too large" error usually comes from nginx, so set `client_max_body_size 11m;`. Convert SVG or HEIC files first.

The page still shows up on Google after hiding it It drops out as Google visits again, usually within a few days or weeks. Search Console's Removals tool is faster.

Link previews have stopped working in WhatsApp or iMessage Hiding the page from crawlers blocks preview bots too. Tick **Still allow link previews in chat apps** under Settings > Privacy. The apps cache previews, so an old link can take a while to update.

`curl https://my-site/` gets a 403 With the page hidden, scripts like curl are refused. Try `curl -A "Mozilla/5.0" https://my-site/`.

Development

Stack

The front end is React 18 built with Vite. It's plain JavaScript with JSDoc types and hand-written CSS using custom properties, with no CSS framework. Drag and drop is `@dnd-kit`, brand icons come from `simple-icons` (CC0) and the fonts are bundled with `@fontsource-variable`. The editor is split into its own bundle, so visitors only download the public page, which is about 100 KB gzipped. The Retro grid background is a separate small chunk as well.

The server is Express 5 on Node.js 22.9 or newer, plus `@dicebear` for the default avatars. Everything is stored in JSON files. Sign-in, sessions, uploads and rate limiting are written by hand on top of Node's `crypto` module rather than pulled in from packages.

```
npm install
npm run dev            # API on :3001, Vite with hot reload on :5173
npm run lint
npm run build          # build the client and package the release zip
npm run build:client   # build the client only
npm run release        # re-package the zip from the existing client/dist
```

`npm run build` writes `release/trellis-<version>/` and `release/trellis-<version>.zip`, with a `.sha256` next to it. The version comes from the root `package.json`, so change that (and the ones in `client/` and `server/`) before a release. The production dependencies are copied out of your own `node_modules`, so run `npm ci` first if you want a clean build.

Code tour

```
client/src/
├── main.jsx, App.jsx           entry point, decides between the page and the editor
├── api/client.js               fetch wrapper, adds the CSRF header
├── lib/
│   ├── tileRegistry.js         every card type: component, sizes, fields, Add menu
│   ├── platforms.js            brand icons and colours, URL to platform, button labels
│   ├── embeds.js               builds embed URLs (Spotify, Apple Music, YouTube, X…)
│   ├── layout.js               card sizes and shapes
│   ├── appearance.js           design presets, fonts, CSS variables
│   ├── avatars.js              DiceBear avatar styles
│   ├── presetFiles.js          saving, exporting and importing presets
│   ├── richText.js             rich text rendering and cleaning
│   └── types.js                JSDoc types for the page document
├── hooks/                      usePageEditor (state, undo, autosave), useFeed, useTheme,
│                               useFavicon, useSiteName, useImageUpload
├── components/
│   ├── PublicPage, NotFoundPage, ProfileHeader, Avatar, TileGrid, Tile, PageBackground, Icons
│   ├── tiles/                  one component per card type
│   └── editor/                 the editor: grid, dialogs, toolbar, design panel
└── styles/                     base.css, tiles.css, editor.css

scripts/package-release.mjs     builds the release zip

server/src/
├── index.js                    app setup, HTTPS, bot blocking, headers, routes, 404
├── config.js                   environment variables
├── routes/                     auth, page, draft, uploads, feeds, integrations…
└── lib/
    ├── validate.js             the schema for everything that gets saved
    ├── store.js                JSON storage
    ├── auth.js                 passwords, sessions, rate limiting, CSRF check
    ├── adminPath.js            the secret editor path
    ├── params.js               query parameters allowed on the page
    ├── blogFeeds.js            RSS and Atom reader for blog cards
    ├── robots.js               crawler lists, robots.txt, noindex
    ├── html.js                 the HTML shell: meta tags, favicon, theme, manifest
    ├── appearance.js           CSS variables applied before first paint
    ├── avatar.js               DiceBear default avatars
    ├── presets.js              server presets (presets/ or PRESETS_DIR)
    ├── safeFetch.js            https client for feeds (public addresses only)
    ├── imageMeta.js            strips location and camera data from uploads
    ├── files.js                atomic file writes
    ├── integrations.js         Instagram token storage and renewal
    └── cache.js                short-lived cache for feeds
```

`tileRegistry.js` and `validate.js` are the two files you'll touch most.

How a page is served

1. A request for `/` goes to `renderShell()`. It takes the built `index.html` and adds the title, meta and OpenGraph tags, the favicon, the robots directives, a small script that sets the theme and colours before anything paints, and the page itself as `window.__TRELLIS_PAGE__`.

2. React renders straight from that, without another request.
3. The editor's secret path gets the same shell with `window.__TRELLIS_EDITOR__ = true` and stricter headers, and `App.jsx` loads the editor bundle.
4. Unknown paths and query parameters get the shell with `window.__TRELLIS_404__ = true` and a 404 status, and `App.jsx` shows `NotFoundPage`. Under the Vite dev server anything other than `/` and the editor shows it.

The page document

```
{
  version: 2,
  profile: { name, bio, location, avatar },
  settings: {
    theme, accent, pageTitle /* site name */, description,
    noindex, blockAI, allowPreviews,
    favicon: { type, emoji, image },
    appearance: { preset, colors, background, cardStyle, radius, font, animation },
  },
  tiles: [{ id, type, w, h, data }],   // w: 1–4 columns, h: 1–8 rows
}
```

`sanitizePage()` upgrades older documents when they're loaded.

Adding a card type

1. Write the component in `client/src/components/tiles/`. It gets `{ tile, shape }`. `shape` is one of `bar`, `sm`, `wide`, `tall` or `lg`, and `tile.w` and `tile.h` give the exact size. `LinkCard` gives you the standard look.
2. Add it to `TILE_TYPES` in `client/src/lib/tileRegistry.js` with a label, icon, the component, `layout` (default, minimum and maximum size), `defaults` and the editor `fields`. Field kinds are `text`, `url`, `email`, `textarea`, `richtext`, `links`, `checkbox`, `select`, `urllist`, `image` and `note`. Then add it to `ADD_MENU`.
3. Add the same fields to `TILE_SCHEMAS` in `server/src/lib/validate.js`. Anything not listed there is thrown away when the page is saved, which catches most people out the first time.
4. If it uses an iframe, add a URL builder to `lib/embeds.js` and the provider's https origin to `FRAME_SOURCES` in `server/src/index.js`.
5. If it shows live data, add a loader to `server/src/routes/feeds.js` and use the `useFeed()` hook in the component.

Adding a platform icon

Add an entry to `PLATFORMS` in `client/src/lib/platforms.js` with a hostname pattern and a simple-icons icon. You can also give it a button label in `PLATFORM_ACTIONS` and an entry in the Add menu with the URL filled in.

Tests

```
npm test             # builds the client, then runs everything (about 20 s)
npm run test:quick   # unit, API and update tests, no build or browser
```

They use Node's built-in test runner. Each suite starts a real server on a free port with an empty temporary data folder.

- `test/unit.test.mjs`: backup archives (checked against the system `tar`), the zip reader, authenticator codes (the RFC 6238 test vectors) and S3 request signing (AWS's own example).
- `test/api.test.mjs`: access control on every editor endpoint, the CSRF header, sign-in only from the editor path, path traversal, script injection into the page, locked, hidden and scheduled cards, the 404 page, two-tab conflicts, short links, backups, two-step sign-in and devices.
- `test/update.test.mjs`: `npm run update` keeps the data, `.env` and presets, checks versions and rolls back.
- `test/browser.test.mjs`: Chrome at phone, tablet and desktop widths (no errors, no sideways scrolling), rich text cleaning, button contrast, editing and publishing, two tabs, and an axe-core accessibility check of the page, the 404 page, the sign-in screen and the editor dialogs.

The browser tests look for `CHROME_PATH`, then Google Chrome, then Edge, and are skipped rather than failed if there's none. GitHub Actions runs the lot on every push (`.github/workflows/test.yml`).

If you fix a bug, add a test that fails without the fix.

Visual checks

Before a release I go through the page and the editor at desktop (1280 to 1920px), tablet (820 and 1024px) and phone (320, 360 and 390px) widths, in a light design, a dark one like Crystal and an outlined one, with some long names and labels. Things to look for: sideways scrolling, text spilling out of a card, cards overlapping, editor controls or dialogs going off screen, and touch selection (one tap selects, a second tap edits). Long text should fade out at the bottom of a card rather than cutting a line in half.

Conventions

- Function components and hooks only. The one exception is the error boundary in `Tile.jsx`.
- Editor state changes go through the `usePageEditor` actions, otherwise undo, redo and autosave stop working.
- No `dangerouslySetInnerHTML`, and user input never goes straight into an iframe `src`.
- Everything that gets saved is validated in `server/src/lib/validate.js`.
- Modules and components get a short comment saying why they exist, not what each line does.

API

✓ means the request needs a session cookie and the `X-Requested-With: trellis` header.

Method	Path	Auth	What it does
GET	`/api/page`		The published page as visitors see it (hidden and scheduled cards removed, locked content left out)
PUT	`/api/page`	✓	Replace and publish the page in one go. Returns the cleaned page
POST	`/api/page/validate`	✓	Clean a page without saving it (used for imports)
GET	`/api/auth/me`		`{ authenticated, passwordSet, twoFactor? }`
POST	`/api/auth/login`		`{ password, path, code? }`, where path is the editor path. Sets the session cookie. With two-step sign-in on and no code, returns 401 `code_required`
POST	`/api/auth/logout`		Ends the session and clears the cookie
GET	`/api/auth/security`	✓	Two-step status, signed-in devices, recent sign-ins
POST	`/api/auth/2fa/setup`	✓	A new secret and `otpauth://` address (not active until enabled)
POST	`/api/auth/2fa/enable`, `/disable`, `/recovery`	✓	Take `{ password, code }`, `{ password, code }` and `{ password }`. Enable and recovery return the `recoveryCodes` once
DELETE	`/api/auth/sessions/:id`	✓	Sign out one device
POST	`/api/auth/sessions/others`	✓	Sign out every other device
GET, POST	`/api/backups`	✓	Backup status, or back up now
GET, POST	`/api/shortlinks`	✓	List short links, or add one (`{ slug, url, note }`)
PUT, DELETE	`/api/shortlinks/:slug`	✓	Change or delete a short link
POST	`/api/uploads`	✓	Raw image or PDF in the body, returns `{ url, size }`
GET, PUT	`/api/draft`	✓	The working copy (`{ page, published, hasChanges, publishedAt, revision }`), or autosave it
POST	`/api/draft/publish`	✓	Make the draft (or the page in the body) live
POST	`/api/draft/discard`	✓	Throw the draft away
GET	`/api/draft/revision`	✓	`{ revision }`, checked when an editor tab gets focus
GET	`/api/versions`, `/api/versions/:id`	✓	Version history, or one version (`?restore=1` saves the draft first)
POST	`/api/collect`		Statistics beacon (`{ type: 'view' | 'click', … }`, sent as text/plain)
GET, DELETE	`/api/stats?days=`	✓	Statistics, or delete them all
POST	`/api/unlock/:tileId`		`{ code }`, returns the content of a locked card
POST	`/api/forms/:tileId/contact`, `/subscribe`		Contact message or newsletter sign-up
GET, PATCH, DELETE	`/api/inbox`, `/api/inbox/:id`	✓	Contact messages: list, mark as read, delete

POST	`/api/geocode`	✓	`{ q }`, returns places from OpenStreetMap Nominatim (for map cards)
GET	`/api/feeds/:tileId`		Live data for Bluesky, Mastodon, Instagram, GitHub and blog cards
GET	`/api/avatar.svg?style=&seed=`		A generated default avatar, cached for a day
GET	`/api/presets`	✓	Design presets installed on the server
POST	`/api/presets/validate`	✓	Clean a preset (used for preset imports)
GET	`/api/integrations`	✓	Which integrations are connected (no secrets)
PUT, DELETE	`/api/integrations/instagram`	✓	Connect (`{ token }`) or disconnect Instagram
PUT, DELETE	`/api/integrations/newsletter`	✓	Connect (`{ provider, apiKey, listId }`) or disconnect a newsletter service
GET	`/api/editor-check?path=`		Whether a path is the editor (used by the dev server)
GET	`/api/health`		`{ ok: true }`. Requests from the server itself to a loopback address also get `version`, never through a proxy

Draft writes (`PUT /api/draft`, publish and discard) can send an `X-Base-Revision` header with the last revision the editor saw. If the draft has changed since then, they return 409 with `{ code: 'conflict', page, published, revision }` instead of overwriting it. Scripts that leave the header out aren't affected.